

Lab & Activities Week 01 LLM Fundamentals

Integrantes:

José Martínez-02220131016

Institución: Universidad de Santander

Pregrado: Ingeniería Se Sistemas

Docente: Jorge Galvis

26 de Agosto 2026

Activity 1

1. Where should information that changes on every single request live?

- ☐ In the system prompt, appended each time
- ☒ In the user message / a dedicated per-call context block
- ☐ It doesn't matter, the model treats both identically

Correct answer: B. Keep persistent behavior in the system prompt and per-call data out of it.

2. Few-shot prompting works mainly by...

- ☐ Permanently updating the model's weights
- ☐ Fine-tuning a small adapter on your examples
- ☒ Letting the model pattern-match the shape of the examples for that single call (in-context learning)

Correct answer: C. No weights change; the examples only affect this one call's context.

3. Chain-of-thought prompting is most worth using for...

- ☒ A multi-step word problem with dependent steps
- ☐ Looking up a single fact
- ☐ Classifying one word as positive or negative

Correct answer: A. On simple lookups/classification, CoT just adds latency and cost.

4. What do `response_mime_type="application/json"` and `response_schema` together guarantee?

- ☐ The response will be short
- ☒ The response will be valid JSON conforming to your schema
- ☐ The response will always have low temperature

4. What do `response_mime_type="application/json"` and `response_schema` together guarantee?

- ☒ The response will be short
- ☐ The response will be valid JSON conforming to your schema
- ☐ The response will always have low temperature

Correct answer: B. Schema forcing constrains structure, not wording or temperature.

Check answers

Score: 4 / 4

Activity 2

System prompt vs no system prompt

1. System prompt vs. no system prompt

Same user question, with and without a system instruction.

```
question = "My bike's front brake feels loose, what should I check?"

no_system = client.models.generate_content(model=MODEL, contents=question)
print("--- Without system prompt ---")
print(no_system.text)

with_system = client.models.generate_content(
    model=MODEL,
    contents=question,
    config=types.GenerateContentConfig(
        system_instruction=(
            "You are a support assistant for a bike rental company. "
            "Only answer questions related to bike rentals, maintenance, and safety. "
            "Keep answers under 3 sentences. If asked something unrelated, politely decline."
        ),
    ),
)
print("\n--- With system prompt ---")
print(with_system.text)
```

[3] ✓ 11.2s Python

... Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.

--- Without system prompt ---

A loose front brake is usually caused by cable slack, worn pads, or air in the hydraulic line. Because your front brake provides about 70% of your stopping power, getting it dialed in is critical.

Here is a step-by-step checklist to diagnose and fix the issue, categorized by common causes:

1. The Quick Checks (Start Here)

- * **Is the quick-release open?**
- * **Rim brakes:** There is often a small lever on the caliper or a noodle (on V-brakes) that you unhook to release the brake.
- * **Disc brakes:** Ensure the wheel axle is seated properly in the fork dropouts.
- * **Is the caliper or lever physically loose?**
- * Check the bolts holding the brake lever to the handlebar and the brake caliper to the fork. Tighten them.

LLM Answer:

"Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream."

--- Without system prompt ---

A loose front brake is usually caused by cable slack, worn pads, or air in the hydraulic line. Because your front brake provides about 70% of your stopping power, getting it dialed in is critical.

Here is a step-by-step checklist to diagnose and fix the issue, categorized by common causes:

1. The Quick Checks (Start Here)

** **Is the quick-release open?***

** Rim brakes: There is often a small lever on the caliper or a noodle (on V-brakes) that you unhook to remove the wheel. If it's open, the brake will feel completely slack.*

** Disc brakes: Ensure the wheel axle is seated properly in the fork dropouts.*

** **Is the caliper or lever physically loose?***

** Check the bolts holding the brake lever to the handlebar and the brake caliper to the fork. Tighten them if they wiggle.*

2. If You Have Cable Brakes (Rim or Mechanical Disc)

If pulling the lever goes all the way to the handlebar, the cable has likely stretched, slipped, or the pads are worn.

** **Check the Barrel Adjuster:***

** Locate the small threaded barrel where the cable enters the brake lever (or caliper).*

** Unscrew it counterclockwise a few turns. This extends the housing and tightens the cable.*

** If this fixes it, tighten the locking ring against the lever body to hold it in place.*

** **Check the Brake Pads:***

** Look at the pads. If the grooves are gone or they are thinner than 1–2 mm, they need to be replaced. Worn pads create a large gap, making the brake feel loose.*

** **Tighten the Cable (Pinch Bolt):***

** If the barrel adjuster isn't enough:*

1. Screw the barrel adjuster almost all the way back in.

2. Loosen the cable pinch bolt on the brake caliper.
3. Pull the cable taut with pliers while pushing the brake arms slightly closed.
4. Retighten the bolt securely and test the lever.

** **Inspect the Cable and Housing:***

** Check for fraying, rust, or kinks. Make sure the black cable housing is seated firmly inside the stops on the frame and lever.*

3. If You Have Hydraulic Disc Brakes

If your brake lever feels "spongy" and pulls all the way to the grip without much resistance, cable adjustments won't apply.

** **Air in the Brake Lines:***

** This is the most common issue. Hydraulic systems are closed; if air gets inside, the lever will feel soft/loose.*

** **Fix:** The brake needs to be ****bled**** (flushing old fluid and removing air bubbles). This requires a specific bleed kit and brake fluid (mineral oil or DOT fluid, depending on the brand).*

** **Piston Retraction / Worn Pads:***

** If the pads are worn thin, the pistons have to travel farther. Inspect the pad thickness.*

** **Fluid Leaks:***

** Check the lever, hose connections, and caliper for any oily residue. If fluid is leaking, the brake will lose pressure and contaminate the pads.*

Summary Checklist:

1. ****Pads:**** Are they worn out?
2. ****Cable:**** Is it loose, slipped, or disconnected? **(Cable brakes)**

3. ****Barrel Adjuster:**** *Can you take up the slack by turning it out? *(Cable brakes)**
4. ****Air/Fluid:**** *Does the system need a bleed or have a leak? *(Hydraulic brakes)**

**Safety Note:* If you adjust the brake and it still doesn't stop the wheel with authority before the lever hits the handlebar, take it to your local bike shop before riding.*

--- With system prompt ---

Check the barrel adjuster on your brake lever and turn it counterclockwise to increase cable tension. Also, inspect the brake cable at the wheel to ensure it hasn't slipped, and verify that the brake pads aren't overly worn. If it still feels unsafe, please bring the bike to one of our rental stations for immediate service."

No system prompt answer: "To fix a loose front brake, the solution depends on whether you have Cable-Actuated Brakes (Rim or Mechanical Disc) or Hydraulic Disc Brakes..." (goes on for a full step-by-step guide, ~400 words, with headers and a safety checklist).

With system prompt answer: "Check the brake cable tension and turn the barrel adjuster near the brake lever to tighten it, while also inspecting the brake pads for wear. If you are riding one of our rentals and the issue persists, please stop riding and visit the nearest station for a free adjustment or bike swap."

On this part I didn't change any code, just run it as is. I see how the answer without the system prompt goes really deep, explaining every brake type like a full mechanic guide, meanwhile with the system prompt the answer get short (2 sentences) and align to the bike-rental context, even adding a business action that wasn't there before ("stop riding and visit the nearest station"). So the system prompt is not just changing the tone, it's cutting the content and adding scope-specific behavior.

Zero-shot vs Few-shot

2. Zero-shot vs. few-shot

Watch the output format stabilize once examples are added.

```
zero_shot = 'Classify the sentiment of this review as positive, negative, or neutral: "It\'s fine, does w  
few_shot = \'\'Classify the sentiment of a review as positive, negative, or neutral.  
  
Review: "Fast shipping and the product works great."  
Sentiment: positive  
  
Review: "It arrived broken and support never replied."  
Sentiment: negative  
  
Review: "It\'s fine, does what it says on the box."  
Sentiment:\'\'  
  
print("Zero-shot:", client.models.generate_content(model=MODEL, contents=zero_shot).text.strip())  
print("Few-shot: ", client.models.generate_content(model=MODEL, contents=few_shot).text.strip())
```

4] ✓ 3.6s Python

```
Zero-shot: **Neutral**  
Few-shot: neutral
```

Generate + Code + Markdown

On this part I also didn't change anything on the code, just run it as is. Both answers get the correct label (neutral), but the format is different: zero-shot come back with markdown bold and capitalized ("**Neutral**"), and few-shot come back plain lowercase ("neutral"), matching exactly the format of the examples I gave in the prompt (positive / negative). This shows that few-shot is not just about getting the right answer, it's about locking the output format so I can parse it later without extra cleanup.

Chain-of-thought vs. direct answer

3. Chain-of-thought vs. direct answer

```
problem = "A store had 120 apples. It sold 35% of them in the morning and 28 more in the afternoon. How m

direct = client.models.generate_content(
    model=MODEL,
    contents=problem + " Answer with just the number.",
    config=types.GenerateContentConfig(temperature=0.1),
)
print("Direct:", direct.text.strip())

cot = client.models.generate_content(
    model=MODEL,
    contents=problem + ' Think step by step, then give the final answer on its own line starting with "An
    config=types.GenerateContentConfig(temperature=0.1),
)
print("\nChain-of-thought:\n", cot.text)
```

[5] ✓ 5.4s Python

... Direct: 50

Chain-of-thought:

1. Calculate the number of apples sold in the morning:
 $35\% \text{ of } 120 = 0.35 \times 120 = 42\$ \text{ apples.}$
2. Add the number of apples sold in the afternoon to find the total sold:
 $42 + 28 = 70\$ \text{ apples.}$
3. Subtract the total number of sold apples from the initial amount:
 $120 - 70 = 50\$ \text{ apples.}$

Answer:
50

Chain-of-thought: "35% of 120 = 42 apples sold in the morning. 42 + 28 = 70 apples sold total. 120 - 70 = 50 apples left. Answer: 50"

On this part both answers land on the same correct result (50), so this time the difference is not the accuracy, it's the explainability. The direct answer just give me the number with no way to check how it get there, meanwhile the chain-of-thought answer show every step of the math (35% calculation, sum of both sales, subtraction from the total), so I can actually verify the reasoning instead of trusting a single token. A direct answer could be right by luck, and I would never know it — with CoT I can catch it if a step is wrong.

Activity 3

At this point I ask to a agent to generate the vague prompt for a real-word task also, I just passed on a different LLM with different responses like in the below image

```
contract_a = """This Lease Agreement is entered into between Northwind Properties,
Jane Rivera ("Tenant") for the property at 88 Elm Street. The lease term is 12 months starting
March 1, 2026, at a monthly rent of $1,450, due on the 1st of each month. A late fee of $75 applies
after the 5th. The Tenant is responsible for utilities except water. Either party may terminate
early with 60 days written notice, subject to a 1-month rent penalty. The security deposit is $1,450,
refundable within 30 days of move-out minus damages."""

contract_b = """This Non-Disclosure Agreement is made between DataCore Analytics Inc. ("Disclosing Party"
and Marcus Webb ("Receiving Party"). The Receiving Party agrees to keep all technical specifications,
source code, and client lists confidential for a period of 3 years from the date of signing. Confidential
information may not be shared with third parties without prior written consent. Breach of this agreement
entitles the Disclosing Party to seek injunctive relief and damages of no less than $50,000. This agreeme
is governed by the laws of the State of Delaware."""

weak_prompt = "Summarize this legal contract."

print("--- BEFORE: weak prompt on Contract A (lease) ---")
print(client.models.generate_content(model=MODEL, contents=f"{weak_prompt}\n\n{contract_a}").text)

print("\n--- BEFORE: weak prompt on Contract B (NDA) ---")
print(client.models.generate_content(model=MODEL, contents=f"{weak_prompt}\n\n{contract_b}").text)
```

✓ 7.4s Python

--- BEFORE: weak prompt on Contract A (lease) ---
Here is a summary of the key terms of the Lease Agreement:

- Parties:** Northwind Properties (*Landlord*) and Jane Rivera (*Tenant*)
- Property:** 88 Elm Street
- Lease Term:** 12 months, beginning March 1, 2026
- Rent:** \$1,450/month, due on the 1st of each month (a \$75 late fee applies after the 5th)
- Security Deposit:** \$1,450, refundable within 30 days of move-out, less any repair/damage costs
- Utilities:** Tenant is responsible for all utilities except water
- Early Termination:** Either party may terminate early with 60 days' written notice and payment of a

--- BEFORE: weak prompt on Contract B (NDA) ---
Here is a summary of the Non-Disclosure Agreement:

- Parties Involved:** DataCore Analytics Inc. ("Disclosing Party") and Marcus Webb ("Receiving Party").
- Confidential Information:** Technical specifications, source code, and client lists.
- Obligations & Duration:** The Receiving Party must keep the information confidential and cannot share it
- Penalties for Breach:** The Disclosing Party is entitled to pursue injunctive relief and recover damages
- Governing Law:** The agreement is governed by the laws of the State of Delaware.

Contract A (lease) came back as 7 bullets: Parties, Property, Lease Term, Rent, Security Deposit, Utilities, Early Termination.

Contract B (NDA) came back as 6 bullets, but with different headers: Parties Involved, Protected Information, Obligations, Duration, Remedies for Breach, Governing Law.

the weak prompt has no role, no output structure, and no examples. It happens to produce readable bullets both times (the model defaults to some structure on its own), but the headers and bullet count are different for every document — there's no fixed schema, so I can't reliably parse "the obligations section" or "the risk section" out of the response, and no way to guarantee the summary calls out risk at all (Contract A doesn't mention any risk framing, Contract B does through

"Remedies for Breach"). This is the actual problem the group workshop is about: missing role + missing output structure = inconsistent shape, even when the content itself is fine.

Activity 4

4. Schema-forced JSON output

4a. Raw SDK

```
class TicketTriage(BaseModel):
    category: str # "billing" | "technical" | "account" | "other"
    urgency: str # "low" | "medium" | "high"
    summary: str

response = client.models.generate_content(
    model=MODEL,
    contents="My card was charged twice for the same order and I need this fixed today.",
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=TicketTriage,
    ),
)
ticket: TicketTriage = response.parsed
print(ticket)
print(ticket.category, "|", ticket.urgency)
```

[8] ✓ 2.2s Python

... category='Billing' urgency='High' summary='Customer was charged twice for the same order and requires same-d: Billing | High

response.parsed already come back as a real TicketTriage instance, not a string I need to `json.loads()` myself. No parsing errors, category and urgency land exactly on values that make sense ("Billing" / "High") given the schema comments as hints. This is the fix for the "unconstrained free-text output breaks the JSON pipeline" problem the instructor showed live response_schema=TicketTriage forces the shape instead of hoping the model wraps things in json fences correctly.

4b. Google ADK — `output_schema` (light preview)

ADK's `LlmAgent` accepts a Pydantic model directly via `output_schema` and returns validated structured output.

```
from google.adk.agents import Agent
from google.adk.runners import InMemoryRunner

triage_agent = Agent(
    model=MODEL,
    name="triage_agent",
    instruction="Triage the support message into the given schema.",
    output_schema=TicketTriage,
)

async def ask_adk_agent(agent, prompt, app_name="week2_app", user_id="student"):
    runner = InMemoryRunner(agent=agent, app_name=app_name)
    session = await runner.session_service.create_session(app_name=app_name, user_id=user_id)
    content = types.Content(role="user", parts=[types.Part.from_text(text=prompt)])
    final_text = None
    async for event in runner.run_async(user_id=user_id, session_id=session.id, new_message=content):
        if event.content and event.content.parts and event.content.parts[0].text:
            final_text = event.content.parts[0].text
    return final_text

raw_json = await ask_adk_agent(triage_agent, "My card was charged twice for the same order")
print(raw_json)
print(TicketTriage.model_validate_json(raw_json))
```

[10] ✓ 3.4s Python

... Direct use of automatic function calling (AFC) in AsyncModels.generate_content is not recommended. The following JSON is a simplified representation of the response: {"category": "Billing", "urgency": "High", "summary": "Customer was charged twice for the same order and"} category='Billing' urgency='High' summary='Customer was charged twice for the same order and'

Same category/urgency as the raw SDK call in 4a, just a slightly reworded summary. The difference here is the ADK `LlmAgent` still hands back a plain JSON string through the event stream (I had to `model_validate_json` it myself), while the raw SDK's `response.parsed` in 4a did that step automatically. So `output_schema` on ADK guarantees the JSON is well-formed, but the raw SDK path is one step shorter to a typed object.

```
# Exercise 1 – Design a few-shot prompt
# Task picked: classify an email as "spam" or "not spam".

zero_shot_template = 'Classify this email as "spam" or "not spam": "{email}"'

few_shot_template = '''Classify an email as "spam" or "not spam".

Email: "You've won a free iPhone! Click here now to claim your prize before it expires!!!
Label: spam

Email: "Hey, just checking if we're still on for the 3pm meeting tomorrow."
Label: not spam

Email: "URGENT: Your account will be suspended. Verify your password immediately at this
Label: spam

Email: "{email}" 🐼
Label:'''

test_emails = [
    "Reminder: your dentist appointment is confirmed for Thursday at 10am.",
    "CONGRATULATIONS!!! You have been selected to receive $5000 cash, reply now with your t",
    "Can you send me the Q3 report before end of day? Thanks.",
]

for email in test_emails:
    zero = client.models.generate_content(model=MODEL, contents=zero_shot_template.format(email=email))
    few = client.models.generate_content(model=MODEL, contents=few_shot_template.format(email=email))
    print(f"Email: {email}")
    print(f"  Zero-shot: {zero!r}")
    print(f"  Few-shot: {few!r}\n")
```

✓ 11.1s

Python

```
Email: Reminder: your dentist appointment is confirmed for Thursday at 10am.
Zero-shot: '**Not spam**'
Few-shot: 'not spam'

Email: CONGRATULATIONS!!! You have been selected to receive $5000 cash, reply now with your t
Zero-shot: '**Spam**'
Few-shot: 'spam'

Email: Can you send me the Q3 report before end of day? Thanks.
Zero-shot: '**Not spam**'
Few-shot: 'not spam'
```

All 6 labels are correct on both prompting styles, so like in section 2, accuracy isn't the story here, format is: zero-shot keeps wrapping the label in markdown bold and capitalizing it ('**Spam**'), few-shot locks it to the exact lowercase word shown in the 3 examples ('spam'). Same pattern holds even for a task I picked myself, not just the sentiment example from section 2, few-shot's main value is consistency of the output shape, not correctness.

```
# Exercise 2 – Mini "resume line" parser

class ResumeLine(BaseModel):
    role: str
    company: str
    years: float

resume_lines = [
    "Senior backend engineer at Northwind Traders for 3.5 years",
    "Product designer, Beacon Studio, 2 years",
    "Worked as a data analyst at Vertex Analytics for six months",
]

for line in resume_lines:
    resp = client.models.generate_content(
        model=MODEL,
        contents=line,
        config=types.GenerateContentConfig(
            response_mime_type="application/json",
            response_schema=ResumeLine,
        ),
    )
    parsed: ResumeLine = resp.parsed
    print(f"Input: {line}")
    print(f"Parsed: {parsed}\n")
```

12] ✓ 7.7s Python

Input: Senior backend engineer at Northwind Traders for 3.5 years
Parsed: role='Senior backend engineer' company='Northwind Traders' years=3.5

Input: Product designer, Beacon Studio, 2 years
Parsed: role='Product designer' company='Beacon Studio' years=2.0

Input: Worked as a data analyst at Vertex Analytics for six months
Parsed: role='data analyst' company='Vertex Analytics' years=0.5

All 3 came back as valid ResumeLine instances with no parsing errors, even though the 3 inputs use different phrasing (comma-separated vs. "at ... for ..." vs. "worked as"). The one that surprised me is the last one: I wrote "six months" in plain English and the model converted it to years=0.5 on its own — the years: float type hint alone was enough to make it do the unit conversion instead of just failing or returning a string.

Github URL with all weeks solutions:

https://github.com/JoseMartinez2611/ai_agentic_class

Activity 6

Corte 1 Project Checkpoint & Independent Work

- Draft the **system prompt** for your Corte 1 assistant (from your Week 1 proposal). Define its role, scope, refusal behavior, and output style.

Done on prompts/system_prompt.txt:

Role: Integral calculus tutor explains steps and cites rules/theorems.

Scope: Definite/indefinite integrals, integration techniques (substitution, integration by parts, partial fractions, trigonometric substitution), applications (area, volume), conceptual questions, and problem generation (numerical or real-world contexts).

Derivatives/limits are only included as an intermediate step, never as the main topic.

Refusal: Rejects non-calculus topics, standalone questions on derivatives/limits, and requests for "just give me the answer without explanation" always redirects the discussion by explaining the scope.

Tone: Friendly, warm, and patient (added later upon your request).

- Decide whether any part of its output needs to be structured (JSON) e.g. if your assistant should also return a confidence score or a source citation alongside its answer.

Decided: yes, structured JSON. Fields: answer, steps, rule, confidence (0.0–1.0), source (citation — null until RAG arrives in Week 4).

- Commit prompts/system_prompt.txt to your repo and update PROPOSAL.md with 2–3 example questions your assistant should be able to answer once RAG is added in Week 4.

prompts/system_prompt.txt created and committed. PROPOSAL.md updated with "Example Questions" section 3 example questions (integral of $x \cdot e^x$, area under $\sin(x)$ from 0 to π , why $+C$). Push to week2-advancement branch, pull request open to main:

<https://github.com/JulianGuardian/Cubik-Lite/pull/1>